



손영진

✉ odso4139@naver.com ☎ 010-3975-5037

🌐 <https://sonyoungjin.netlify.app/> 🌐 github.com/sonprogrammer

PROFILE

FSD 아키텍처 기반의 구조적 설계와 AI API 최적화에 강점을 가진 프론트엔드 개발자 손영진입니다. O2O 서비스 및 B2B 관리자 대시보드를 구축하며 FSD(Feature-Sliced Design) 아키텍처를 도입해 모듈 간 의존성을 줄이고 유지보수하기 쉬운 코드 구조를 설계했습니다. 서비스의 특성에 맞춰 전역/서버 상태 관리를 엄격히 분리하고 데이터 캐싱 전략을 통해 유저 경험과 성능을 끌어올리는 데 집중합니다.

• 핵심 역량 및 성과 요약

- B2B 대시보드 & 실시간 데이터 구축: Refine/Antd 기반 어드민 시스템 구축 및 Supabase Realtime을 활용한 실시간 현황 동기화 레이어 설계
- AI API 연동 및 비용 최적화: Groq/Gemini AI 응답의 JSON 형식으로 구조화하고, DB 캐싱으로 API 호출 비용 67% 절감
- 웹 퍼포먼스 및 UX 최적화: REST/WebSocket 하이브리드 수신 전략으로 초기 로딩 속도 80% 단축, LCP 2.0 초 이내 및 CLS 0.001 이하 달성

SKILLS

Front-end

- JavaScript
- TypeScript
- React
- Next.js
- Tailwind CSS
- TanStack-Query
- Zustand
- Recoil

Back-end

- Node.js
- Express.js
- MongoDB
- Supabase

PROJECTS

Mungpass - 개인 프로젝트

01/2026 - 05/2026

애견카페 통합 O2O 및 B2B 관리 대시보드

기술스택: Next.js, TypeScript, Supabase(Auth, Realtime, DB), TanStack Query, Zustand, Tailwind CSS

구조: FSD(Feature-Sliced Design) 아키텍처 적용

주요 성과 및 트러블슈팅

- QR 기반 실시간 입장 동기화
 - Supabase Realtime과 TanStack Query를 연동하여 QR 스캔 즉시 사장님 대시보드에 이용 현황이 새로고침 없이 반영되는 O2O 파이프라인 구축
- FSD 아키텍처 기반 모듈화
 - 레이어별 단방향 의존성을 강제하여 비즈니스 로직과 UI를 분리하고 기능 확장 시 기존 모듈 수정 없는 재사용성 확보로 회귀 테스트 비용 절감
- AI 매출 리포트 & 비용 최적화
 - Gemini AI 기반 매출 분석 자동화 구축 및 결과를 DB에 캐싱하여 API 호출 비용 67% 절감
- 역할 기반 접근 제어(RBAC) 및 보안
 - Supabase RLS 정책과 라우트 가드를 결합하여 사장님/일반 유저/관리자 권한별 데이터 격리 및 비인가 접근 차단

• 웹 퍼포먼스 및 모바일 UX 최적화

- Service Worker PWA 구축, next/image 리소스 우선순위 조정으로 LCP 2.0초 이내, CLS 0.001 이하 달성 및 iOS 뷰포트 인풋 줌 버그 전역 해결

Fuelly - 개인 프로젝트

12/2025 - 01/2026

AI 기반 개인 맞춤 영양 관리 웹 서비스

기술스택: Next.js, TypeScript, Zustand, TanStack Query, Tailwind CSS, Recharts, Groq AI API

주요 성과 및 트러블슈팅

- **AI 응답 데이터 정형화**
 - AI 응답의 형식 편차로 인한 파싱 오류를 줄이기 위해 Groq의 JSON Object 응답 형식을 적용하고 서버에서 응답을 파싱해 구조화된 영양 정보로 반환했습니다.
- **AI Latency 대응 UX 구축**
 - AI 추론 대기 시간(2~3초) 동안 사용자 이탈을 방지하기 위해 스켈레톤 UI 및 상태 기반 로딩 인터랙션을 적용하여 체감 대기 시간 단축
- **보안 중심 인증 아키텍처**
 - Access Token(메모리)과 Refresh Token(HttpOnly Cookie) 기반 이중 보안 설계 및 Next.js Edge Middleware 기반 선제적 라우트 보호
- **Axios Interceptor 기반 동시성 제어**
 - 파편화된 인증 로직을 일원화(중복 코드 70% 감축)하고 토큰 재발급 대기 큐(Queue)를 구현하여 다중 요청 환경에서 서버 부하 최소화
- **서버/전역 상태 분리**
 - TanStack Query(서버 데이터 캐싱)와 Zustand(전역 UI 상태)를 엄격히 분리하여 불필요한 네트워크 요청 및 리렌더링 차단

VC(Virtual Coin) - 개인 프로젝트

03/2025 - 04/2025

Upbit API 기반 실시간 모의 코인 투자 서비스

기술스택: React, TypeScript, Node.js, Express, MongoDB, TanStack Query, Recoil, Upbit API, WebSocket(ws), Lightweight Charts, Tailwind CSS

주요 성과 및 트러블슈팅

- **하이브리드 시세 수신 파이프라인**
 - REST API로 초기 시세를 선 렌더링 후 WebSocket 백그라운드 동기화로 전환하여 초기 시세 로딩 속도를 80% 단축 (3.3초 → 0.7초)
- **실시간 렌더링 과부하 방지**
 - Upbit WebSocket 수신 시 lodash.throttle 전략을 적용하여 과도한 상태 업데이트를 제어하고 Chart.js 차트 반응성 확보
- **Node.js 백엔드 프로덕시 구축**
 - CORS 에러 및 Upbit API 요청 제한(Too Many Requests) 이슈를 중계 서버 구축을 통해 해결

BNTY(Be Next To You) - 개인 프로젝트

2024.09 - 10/2024

트레이너-회원 1:1 맞춤 소통 플랫폼

기술스택: React, TypeScript, Express, Node.js, MongoDB, TanStack Query, Recoil, Socket.IO, Tailwind CSS, PWA

주요 성과 및 트러블슈팅

- **QR 기반 수강권 차감 자동화**
 - 수기 정산의 불투명성을 해결하고자 QR 스캔 시 출석 데이터 및 잔여 수강 횟수가 즉시 차감·동기화되는 오프라인 연결 시스템 설계
- **실시간 채팅 및 읽음 상태 처리**

- WebSocket(Socket.io)을 활용한 1:1 채팅 구축 및 한글 IME 입력 조합 중복 전송 이슈 분기 처리
- 초기 로딩 퍼포먼스 개선
 - React.lazy / Suspense 기반 코드 스플리팅 및 API 병렬 처리(Promise.all)로 LCP 지표 개선 (8.0초 → 1.14초)

Mingle - 팀프로젝트

11/2023 - 11/2023

음악 취향 공유 SNS 서비스 (팀 5인)

기술스택: React, TypeScript, Recoil, TanStack Query, Tailwind CSS, Axios

주요 역할 및 협업

- OpenWeather API CORS 이슈 해결
 - 공통 Axios 인스턴스의 Authorization 토큰 헤더 충돌 문제를 별도 unauthenticated 인스턴스로 분리하여 Preflight 에러 해결
- 서버 데이터 캐싱 전략 적용
 - TanStack Query 도입으로 플레이리스트 등 조회 빈도가 높은 API 데이터의 불필요한 네트워크 재요청 최소화
- 애자일 기반 데일리 스크럼
 - 백엔드 팀원과 API 명세 변경사항을 매일 싱크하고, ESLint/Prettier 공통 규칙을 준수하여 연동 이슈 및 코드 수정 최소화

자기소개서 | 손영진

개발 철학 및 핵심 기술 역량

[데이터 흐름을 설계하고, 사용자 경험으로 연결하는 프론트엔드 개발자]

저는 프론트엔드 개발을 "서버의 데이터와 비즈니스 기능을 사용자가 가장 편리하게 이용할 수 있는 유저 경험으로 연결하는 핵심 인터페이스"라고 정의합니다. 항공소프트웨어공학 전공 및 부트캠프 과정, 그리고 다수의 프로젝트를 거치며 이러한 기술적 지향점을 구체화해 왔습니다.

첫 프로젝트인 PT 관리 서비스(BNTY)를 기획하고 개발할 당시, 프론트엔드에만 머물지 않고 Node.js와 MongoDB 기반의 백엔드 및 DB 구조까지 직접 구축했습니다. 이 과정에서 하나의 서비스 전체를 관통하는 데이터 흐름과 시스템 생애주기를 깊이 있게 이해할 수 있었습니다. 이후 오프라인 O2O 관리 플랫폼(Mungpass) 및 AI 영양 관리 서비스(Fuelly) 등을 개발하며 기술적 범위를 지속적으로 확장했습니다. 서비스의 규모가 커짐에 따라 발생하는 코드 복잡도와 상태 엉킴 현상을 해결하기 위해 FSD(Feature-Sliced Design) 아키텍처를 적극 도입했습니다. 비즈니스 기능별로 슬라이스를 분리해 컴포넌트 간 결합도를 낮추고, 서버 상태(TanStack Query)와 전역 UI 상태(Zustand)를 엄격히 분리함으로써 유지보수성과 확장성을 갖춘 시스템 아키텍처를 설계했습니다. 또한 Groq, Gemini 등 AI API를 연동할 때는 비정형 응답의 JSON 파싱 정형화 파이프라인 및 DB 캐싱을 도입하여 API 호출 비용을 절감하는 등, 단순 화면 작성을 넘어 비즈니스 효율과 퍼포먼스를 동시에 고려하는 개발을 지향합니다. 문제가 발생했을 때 원인을 끝까지 추적해 해결하는 끈기를 바탕으로 목표한 일정 안에서 가장 안정적이고 확장 가능한 최적의 코드를 작성하는 데 집중하고 있습니다.

핵심 프로젝트 & 트러블슈팅 경험

[문제 해결 중심의 아키텍처 설계와 수치로 증명한 성능 개선]

저는 서비스를 개발하며 마주치는 기술적 한계와 운영상의 불편함을 기술적 선택을 통해 구조적으로 해결하는 데 집중해 왔습니다.

O2O B2B 관리 플랫폼인 'Mungpass'를 기획하고 개발할 당시, 사장님 전용 대시보드의 기능 이 확장됨에 따라 컴포넌트 간의 의존성이 복잡해지고 유지보수가 어려워지는 문제를 겪었습니다. 또한 Gemini AI를 활용한 매출 분석 리포트를 생성할 때 유저가 페이지를 조회할 때마다 불필요한 API 호출이 발생하여 운영 비용 증가와 응답 지연이 우려되었습니다.

이를 해결하기 위해 프로젝트 전체에 FSD(Feature-Sliced Design) 아키텍처를 도입했습니다. 비즈니스 기능별로 레이어를 엄격히 나누어 컴포넌트 간 결합도를 낮추고 UI 로직과 비즈니스 로직을 분리하여 추후 새로운 기능을 추가하더라도 기존 모듈의 수정 없이 재사용할 수 있는 구조를 만들었습니다. 더불어 AI 분석 결과를 Supabase DB에 저장하고 사용자 일일 평균 조회 패턴에 맞춰 재사용하는 DB 캐싱 구조를 설계했습니다. 그 결과 불필요한 AI API 호출을 약 67% 절감할 수 있었으며 리소스 우선순위 조정을 통해 LCP 2.0초 이내, CLS 0.001 이하라는 시각적 안정성을 확보했습니다.

한편 AI 맞춤 영양 관리 서비스인 'Fuelly'에서는 Groq AI 연동 시 전달되는 비정형 텍스트 응답으로 인해 UI 렌더링 에러가 발생하는 이슈가 있었습니다. 이를 안정적으로 다루기 위해 서버 단에서 AI 응답을 규격화된 JSON Schema로 파싱 및 검증하여 시스템 데이터와 동기화하는 파이프라인을 직접 구축했습니다. 또한 20여 개의 API 파일에 파편화되어 있던 인증 로직을 Axios Interceptor로 일원화하고 토큰 재발급 대기 큐(Queue)를 활용한 동시성 제어를 구현하여 중복 코드를 70% 이상 감축하고 다중 요청 환경에서도 안정적인 인증을 유지하도록 최적화했습니다.

이처럼 서비스의 확장성과 비즈니스 효율을 함께 높일 수 있는 아키텍처적 대안을 고민하고 실행에 옮기는 개발자입니다.

팀워크 및 협업 프로세스

[맥락을 이해하고 원활한 조율을 이끄는 유연한 소통]

저는 개발 과정에서 혼자만의 기술적 완성도에 집착하기보다 디자이너, 백엔드 개발자 등 다른 직군의 관점과 맥락을 이해하고 신뢰를 바탕으로 협업하는 것을 가장 중요하게 생각합니다.

부트캠프 팀 프로젝트로 음악 취향 공유 SNS인 'Mingle'을 개발할 당시 기획 변경과 API 명세 수정이 빈번하게 발생하는 상황을 경험했습니다. 이로 인한 개발 일정 지연과 코드 재작업을 최소화하기 위해 매일 아침 데일리 스크럼 미팅을 제안하고 적극적으로 주도했습니다. 백엔드 팀원과 구현하고자 하는 화면 흐름을 사전에 공유하고 API 명세서의 데이터 구조와 예외 처리 방식을 미리 맞춘 덕분에 프론트엔드와 백엔드 간 연동 이슈를 사전에 차단하고 계획된 일정 내 프로젝트를 완료할 수 있었습니다.

또한 외부 Weather API 연동 과정에서 공통 Axios 인스턴스의 토큰 헤더로 인해 CORS Preflight 에러가 발생했을 때 네트워크 탭과 공식 문서를 분석하여 원인을 직접 파악했습니다. 해당 요청에 한해서만 토큰이 포함되지 않는 기본 인스턴스를 분리 적용하는 대안을 제시하여 문제를 빠르게 해결했던 경험이 있습니다.

팀 프로젝트뿐만 아니라 개인 프로젝트를 진행할 때도 '미래의 동료'가 읽을 코드'라는 마음이 짐으로 작용합니다. FSD 아키텍처를 도입해 코드 구조의 예측 가능성을 높이고 Git commit 컨벤션 문화를 준수하려고 노력합니다. 입사 후에도 서비스의 방향성을 정확히 이해하고 동료들과 적극적으로 소통하며 팀 전체의 생산성에 기여하는 신뢰받는 개발자가 되겠습니다.

지원 동기 및 입사 후 포부

[확장 가능한 아키텍처와 기술적 근거로 서비스 고도화에 기여]

저는 사용자가 마주하는 접점을 직접 설계하는 프론트엔드 개발자로서 서비스가 성장함에 따라 발생하는 기술적 복잡도를 안정적으로 제어하고 비즈니스 가치를 높이는 데 깊은 관심을 가져왔습니다. 특히 오프라인 현장의 문제를 기술로 전환한 O2O 플랫폼(Mungpass)과 AI 기술을 접목한 영양 관리 서비스(Fuelly)를 기획하고 개발하며 실시간 데이터 동기화와 AI 응답 파이프라인 최적화가 서비스 경쟁력에 얼마나 직결되는지 체감했습니다. 이러한 저의 경험과 기술적 방향성은 복잡한 비즈니스 로직을 다루며 AI 기술을 통해 유저 경험을 혁신하고자 하는 귀사의 지원 동기가 되었습니다.

입사 초기에는 귀사의 개발 환경과 코딩 컨벤션에 빠르게 적응하여 맡은 기능과 화면 요구사항을 차질 없이 구현하겠습니다. 프로젝트 진행 과정에서 축적한 FSD(Feature-Sliced Design) 아키텍처 기반의 모듈화 경험과 Zustand, TanStack Query를 활용한 서버/전역 상태 분리 노하우를 바탕으로 기획 변경이나 요구사항 추가에도 유연하게 대응할 수 있는 코드를 작성하겠습니다.

중장기적으로는 프론트엔드 퍼포먼스와 비즈니스 효율을 함께 높이는 기술적 중심축으로 성장하겠습니다. AI API 연동 시 발생할 수 있는 Latency 대처, DB 캐싱을 통한 호출 비용 최적화, 그리고 B2B 대시보드의 실시간 데이터 처리 경험을 활용하여 서비스의 안정적인 고도화에 기여하겠습니다. 나아가 기술적 선택의 이유를 명확한 근거로 설명하며 타 직군 동료들과 유연하게 소통하는 신뢰받는 동료가 되겠습니다.

EDUCATION

한서대학교
항공소프트웨어공학과

02/2024
졸업

주요 학습 및 경험

- **프론트엔드 핵심역량 습득**
 - React 및 TypeScript 기반의 웹 개발 생태계 이해 및 상태 관리 기법 습득
- **실무형 애자일 협업 프로세스**
 - 데일리 스크럼 미팅, 백엔드 팀원과의 소통을 통한 팀 프로젝트 2건 완수
- **Git 기반 협업 체계**
 - Issue 발행, 브랜치 전략(Git Flow) 및 Pull Request 기반 코드 리뷰 문화 적용

CERTIFICATES

- 정보처리기사
- GTQ 1급